



Infrastructure as Code

automate provisioning & configuring infrastructure

<u>creating from scratch</u>	<u>resources exist</u>
<u>terraform</u>	ansible

Introduction



- Infrastructure as Code (IaC) is a DevOps practice where infrastructure (servers, networks, load balancers, etc.) is **defined, managed, and provisioned through code**, instead of manually configuring it through a UI or CLI
- You describe your infrastructure in **declarative or imperative** code, and tools automatically apply it
- Without IaC
 - You manually:
 - Create an EC2 instance in AWS Console.
 - Install NGINX manually via SSH.
 - Configure load balancer manually.
- With IaC
 - You write:
 - # terraform main.tf or ansible playbook
 - resource "aws_instance" "web" {
 - ami = "ami-123456"
 - instance_type = "t2.micro"
 - }

Benefits

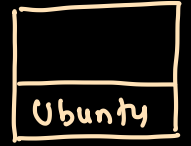


- Consistency: Same configuration every time → eliminates "it works on my machine" issues
- Version Control: Code stored in Git; changes can be reviewed & audited
- Automation: Infrastructure can be deployed/recreated automatically
- Scalability: Same config can spin up 100s of instances easily
- Disaster Recovery: Recreate entire infrastructure from code

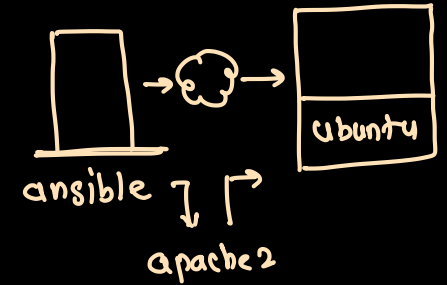
Idempotency

- Idempotency means – running the same code multiple times leads to the same result.
- In IaC, this ensures stability and predictability:
 - “Running your Ansible playbook 5 times should leave your infrastructure in the same desired state”
- If you run it:
 - First time → Installs nginx ✓
 - Second time → Checks, finds it's already installed → Does nothing
- That's idempotent behavior.
- Why it matters:
 - Prevents duplicate work or resource creation
 - Makes automation safe to re-run.
 - Allows recovery from partial failures easily

```
- name: Install nginx
  apt:
    name: nginx
    state: present
```



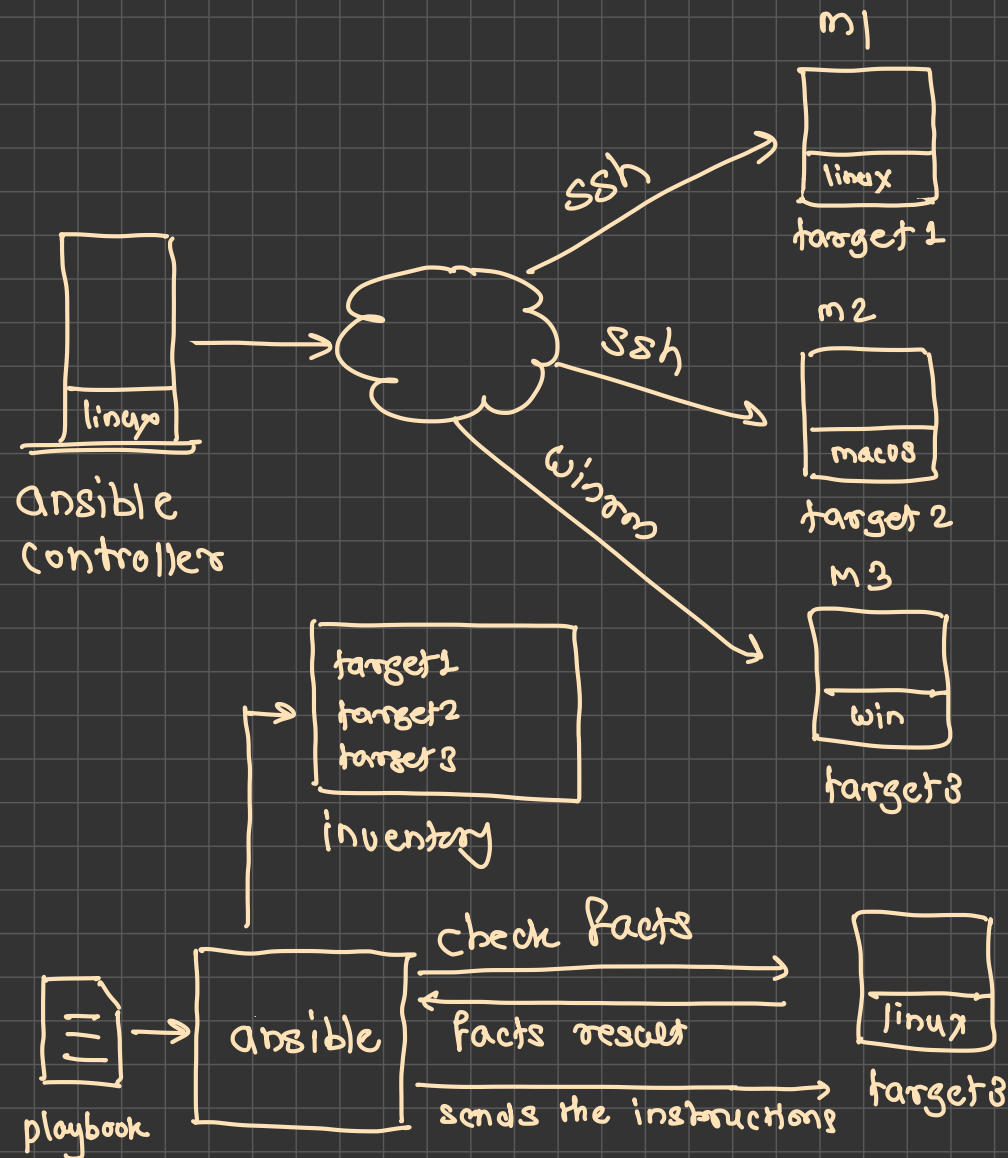
sudo apt install apache2 ~Y
script.sh → Not idempotent



push architecture : ansible, saltstack

Agentless → No agents are running on target machines

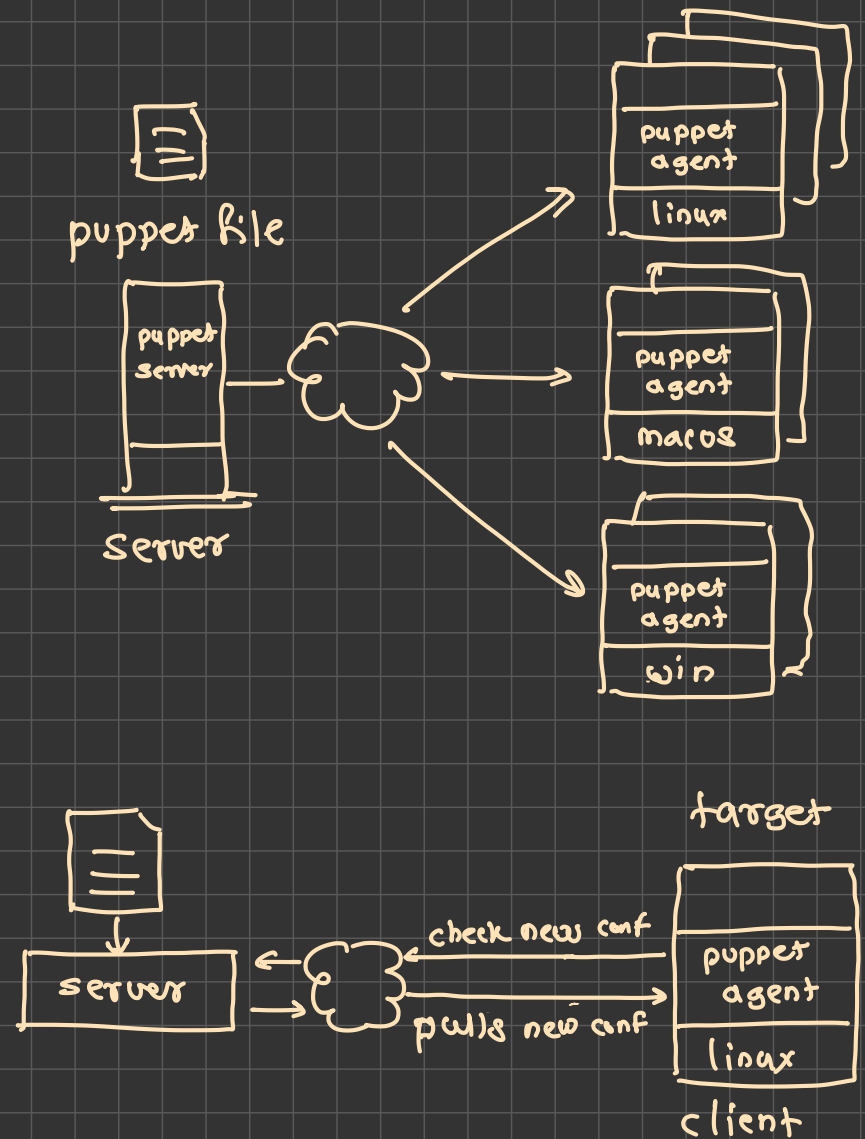
→ No ansible installation is required



pull architecture : puppet, chef, saltstack

→ agents must be installed & running in the target machines

→ agent pulls config from server



Architectures of IaC Tools — Push vs Pull



- Push-based Architecture → *Agentless*
- The controller/master pushes configuration to target systems.
- Communication is initiated from the master.
- Examples
 - Ansible
 - SaltStack (in push mode)
- Workflow
 - Admin runs ansible-playbook site.yml. *winRM*
 - Ansible (control node) connects to servers (usually via SSH).
 - Pushes the configuration and applies changes.
- Pros
 - Simple to understand
 - Real-time control *targets*
 - No agent required on servers
- Cons
 - Harder to scale for 1000s of nodes
 - Requires network access from control node to all targets

- Pull-based Architecture
- Each agent on the managed node pulls configuration from a central server at intervals.
- Communication is initiated from the agent.
- Examples
 - Puppet
 - Chef
 - SaltStack (in pull mode)
- Workflow
 - Configuration stored on master node
 - Agents periodically check in:
 - puppet agent --test
 - Agent downloads config & applies it
- Pros
 - Scales well
 - Nodes self-manage even if master is busy
 - Easier to operate in secure networks
- Cons
 - Changes are not immediate (depends on pull interval)
 - Slightly more complex setup (requires agents)



Ansible

Introduction



- Ansible is an open source IT automation engine that automates provisioning, configuration management, application deployment, orchestration, and many other IT processes
- It is free to use, and the project benefits from the experience and intelligence of its thousands of contributors
- Ansible provides open-source automation that reduces complexity and runs everywhere
- Using Ansible lets you automate virtually any task
- Ansible uses simple, human-readable scripts called playbooks to automate your tasks
- You declare the desired state of a local or remote system in your playbook
- Ansible ensures that the system remains in that state

→ YAML

→ on-prem / cloud
→ OS → macOS / linux / win

desired state

[apache → present]

Key Features of Ansible



- Agentless Architecture → target must allow passwordless auth
 - Ansible does not require any software or agents to be installed on the target systems. It uses SSH (or WinRM for Windows) to communicate and perform tasks
- Simple YAML Syntax → playbook
 - Playbooks, which define automation workflows, are written in YAML—a human-readable markup language. This makes it easy for teams to understand and collaborate
- Idempotency
 - Ansible ensures that tasks are executed in a way that does not alter the system if it's already in the desired state. This prevents unintended changes
- Cross-Platform
 - Ansible supports multiple platforms, including Linux, Windows, macOS, and network devices like switches and routers
- Extensibility
 - Ansible has a wide variety of modules for interacting with cloud services (AWS, Azure, Google Cloud), databases, storage, and more. Users can also write custom modules
- Inventory Management
 - Ansible uses inventory files to define groups of hosts and their associated variables, making it easy to manage infrastructure at scale
- Event-Driven
 - With tools like Ansible Tower or AWX, you can create workflows that trigger actions based on events, integrating seamlessly with CI/CD pipelines

Common Use Cases



- Provisioning
 - Automating the setup of servers, VMs, or containers
- Configuration Management
 - Ensuring that systems are configured consistently across an environment
- Application Deployment
 - Simplifying the deployment process by automating tasks such as database migrations, package installations, and configuration file updates
- Orchestration
 - Coordinating the interactions between various services, applications, and environments
- Security and Compliance
 - Enforcing policies, applying patches, and performing audits

Why Choose Ansible?



■ Simple Learning Curve

- Uses readable YAML syntax
- No programming background required
- Extensive documentation and community support

■ Agentless Architecture

- No software needed on target systems
- Uses existing SSH infrastructure
- Lower overhead and maintenance

■ Large Module Library

- 3000+ built-in modules
- Covers most common automation tasks
- Community-contributed modules available

■ Cost-Effective

- Open-source and free
- No licensing fees
- Minimal infrastructure requirements

■ Security

- No additional ports needed
- Encrypted communication via SSH
- Ansible Vault for secrets management

■ Scalability

- Can manage thousands of nodes
- Parallel execution capabilities
- Works well with cloud infrastructure

Architecture Components



- Control Node

- The machine where Ansible is installed and executed
- It contains the Ansible engine and executes playbooks to manage the infrastructure
- Typically, this is a Linux-based system as Ansible is developed in Python

- Managed Nodes

- The target systems that Ansible controls (e.g., servers, VMs, containers, network devices)
- Managed nodes do not require any agent software or Ansible installation. Communication is established via
 - SSH: For Linux-based systems
 - WinRM: For Windows systems

- Inventory

- A file or dynamic source that lists the managed nodes (hosts) and organizes them into groups
- Can include variables specific to each host or group
- Default location: /etc/ansible/hosts
- Can be static (text file) or dynamic (e.g., querying a cloud provider's API)

Architecture Components



■ Modules

- Units of work that Ansible executes on the managed nodes
- Examples: File operations, service management, package installations
- Modules are typically written in Python, but they can be any executable script

■ Plugins

- Extend Ansible's functionality. Common types include
 - Action Plugins: Modify how tasks are executed
 - Connection Plugins: Define how Ansible connects to hosts (e.g., SSH, WinRM)
 - Callback Plugins: Handle task outputs and notifications
 - Lookup Plugins: Retrieve data from external sources

■ Playbooks

- YAML files that define automation workflows, specifying the tasks to execute
- Each playbook contains
 - Plays: Define a series of tasks applied to a group of hosts
 - Tasks: Individual actions executed by modules

Architecture Components



- **Facts**

- System information gathered from managed nodes (e.g., OS type, IP addresses)
- Collected using the setup module
- These facts can be used as variables in playbooks

- **Vault**

- A feature to encrypt sensitive data like passwords or API keys
- Encrypted data can be securely included in playbooks or inventory files

- **Execution Environment (Optional)**

- A containerized environment for executing Ansible tasks, ensuring consistency across different systems

Ansible Workflow

■ Prepare the Inventory

- Define the hosts to manage in the inventory file, grouping them as needed

■ Write Playbooks

- Create YAML playbooks to define the desired tasks

■ Execute Playbooks

- Use the ansible-playbook command to run the playbook against the inventory

■ Connect to Managed Nodes

- Ansible uses SSH or WinRM to connect to target systems
- No agent installation is needed, simplifying the setup

■ Task Execution

- Modules specified in the playbook are executed on the managed nodes
- Tasks are idempotent, ensuring the system reaches the desired state without unintended changes

■ Report Results

- Ansible collects the results from the managed nodes and reports back to the control node

```
[group]
< ip address 1 >
< ip address 2 >
```

inventory.ini

targets

Installation



For Ubuntu/Debian

```
> sudo apt update
```

```
> sudo apt install ansible
```

For RHEL/CentOS

```
> sudo yum install ansible
```

For macOS

```
> brew install ansible
```


Inventory Basics



- Ansible inventory is a fundamental component that specifies the hosts (managed nodes) that Ansible will interact with
- It defines the infrastructure that Ansible automates and organizes hosts into groups, enabling efficient management of large-scale environments
- **Key Features of Ansible Inventory**
 - **Hosts**
 - Individual machines or nodes that Ansible manages
 - Identified by IP addresses, hostnames, or FQDNs
 - **Groups**
 - Hosts can be organized into groups for easier management
 - Tasks can target specific groups, enabling group-wide automation
 - **Variables**
 - Metadata or configuration data associated with hosts or groups
 - Example: OS type, ports, or paths to configuration files
 - **Static and Dynamic Inventory**
 - Static Inventory: A simple text file that manually lists hosts and groups.
 - Dynamic Inventory: A script or plugin that dynamically fetches inventory data (e.g., from cloud providers like AWS, Azure, or GCP).

Inventory



■ Static Inventory

- A static inventory is a text file, typically located at `/etc/ansible/hosts`
- It lists hosts and groups in an easy-to-read format

```
# Single host
webserver ansible_host=192.168.1.10
```

```
# Group of hosts
[webservers]
web1 ansible_host=192.168.1.11
web2 ansible_host=192.168.1.12
```

■ Dynamic Inventory

- Dynamic inventory is generated by a script or plugin that queries an external source to populate the inventory
- This is useful for environments where infrastructure changes frequently (e.g., cloud platforms)
- **Key Features**
 - Supports cloud providers like AWS, Azure, Google Cloud.
 - Reduces manual effort by fetching inventory automatically.
 - Maintains up-to-date lists of hosts.

Playbook



- An **Ansible playbook** is a YAML file that defines a series of automation tasks to be executed on managed nodes
- Playbooks are central to Ansible's automation capabilities, allowing you to orchestrate complex processes in a simple, human-readable format
- **Key Features of Ansible Playbooks**
 - **Declarative**
 - You define the desired state of the system, and Ansible ensures the tasks achieve that state
 - **YAML Syntax**
 - Playbooks are written in YAML, a straightforward, human-readable markup language
 - **Idempotent**
 - Tasks are idempotent, meaning they only make changes if the system's state doesn't match the desired state
 - **Reusability**
 - Playbooks can be reused across environments with variable substitution, making them versatile



Playbook structure

- A playbook typically consists of the following components
- **Hosts**
 - Specifies the target hosts or groups defined in the inventory
- **Tasks**
 - Individual units of work, such as installing software, copying files, or restarting services
- **Modules**
 - Built-in or custom functionality executed in tasks (e.g., yum, apt, copy, file)
- **Variables**
 - Dynamic values that can be substituted into tasks (e.g., paths, configurations)
- **Handlers**
 - Tasks triggered by other tasks to perform actions like restarting a service
- **Roles (Optional)**
 - Organized collections of tasks, variables, templates, and handlers, promoting modularity
- **Conditionals and Loops**
 - Control structures to execute tasks based on conditions or iterate over lists

Playbook Example



- name: Install and configure Nginx

hosts: webservers

become: true

vars:

http_port: 80

tasks:

- name: Install Nginx

yum:

name: nginx

state: present

- name: Ensure Nginx is running

service:

name: nginx

state: started

enabled: true



Common Modules



File and Directory Management

- **file:** Manage file and directory properties (e.g., permissions, ownership, creation)
- **copy:** Copy files from the control node to managed nodes
- **fetch:** Retrieve files from managed nodes to the control node
- **template:** Apply Jinja2 templates to files before copying them

```
- name: Ensure directory exists
  file:
    path: /path/to/directory
    state: directory
```

```
- name: Copy a file
  copy:
    src: /path/to/source
    dest: /path/to/destination
```

```
- name: Fetch logs
  fetch:
    src: /var/log/syslog
    dest: /local/path/
```

```
- name: Deploy configuration template
  template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
```



Package Management

- **yum** (RHEL/CentOS) or **apt** (Ubuntu/Debian): Install or manage packages
- **pip**: Manage Python packages
- **dnf**: For Fedora and newer Red Hat-based distributions

```
- name: Install Nginx
  yum:
    name: nginx
    state: present
```

```
- name: Install a package using dnf
  dnf:
    name: httpd
    state: present
```

```
- name: Install Flask
  pip:
    name: flask
```


Service Management



- **service:** Manage system services
- **systemd:** Manage services specifically on systemd-based systems

```
- name: Ensure Nginx is running
  service:
    name: nginx
    state: started
    enabled: true
```

```
- name: Restart a service
  systemd:
    name: apache2
    state: restarted
```



User and Group Management

- **user:** Manage user accounts
- **group:** Manage groups

```
- name: Create a user
  user:
    name: johndoe
    state: present
```

```
- name: Create a group
  group:
    name: developers
```

Networking



- **firewalld**: Manage FirewallD rules on RHEL-based systems
- **ufw**: Manage UFW (Uncomplicated Firewall) on Ubuntu
- **uri**: Interact with web services or APIs

```
- name: Open HTTP port
firewalld:
  port: 80/tcp
  state: enabled
  permanent: true
```

```
- name: Check a web service
uri:
  url: http://example.com
  return_content: yes
```

```
- name: Allow HTTP traffic
ufw:
  rule: allow
  port: 80
  proto: tcp
```

Database Management



- `mysql_db`: Manage MySQL databases
- `postgresql_user`: Manage PostgreSQL users

```
- name: Create a MySQL database
mysql_db:
  name: my_database
  state: present
```

```
- name: Add a PostgreSQL user
postgresql_user:
  name: db_user
  password: secret
```

Cloud and Virtualization



- **ec2**: Manage AWS EC2 instances
- **azure_rm**: Manage Azure resources
- **docker_container**: Manage Docker containers

```
- name: Create an Azure VM
  azure_rm_virtualmachine:
    name: myVM
    resource_group: myResourceGroup
```

```
- name: Launch an EC2 instance
  ec2:
    instance_type: t2.micro
    image: ami-123456
    region: us-east-1
```

```
- name: Start a Docker container
  docker_container:
    name: my_container
    image: nginx
    state: started
```



Task Control and Execution

- **command:** Run commands on managed nodes (not idempotent)
- **shell:** Execute shell commands (supports pipes, redirection, etc.)
- **raw:** Execute raw commands (useful for bootstrapping systems without Python)

```
- name: Run a command  
  command: ls -l /var/log
```

```
- name: Run a shell script  
  shell: ./script.sh
```

```
- name: Bootstrap Python  
  raw: yum install -y python3
```

Others



- **cron:** Manage cron jobs
- **git:** Clone or manage Git repositories

```
- name: Schedule a daily backup
  cron:
    name: "daily backup"
    hour: 2
    job: "/path/to/backup.sh"
```

```
- name: Clone a repository
  git:
    repo: https://github.com/example/repo.git
    dest: /path/to/repo
```



System Information and Debugging

- **setup**: Gather facts about managed nodes
- **debug**: Print messages or variables for debugging

```
- name: Debug a variable
  debug:
    var: my_variable
```

```
ansible all -m setup
```




- **Ansible Galaxy** is a platform and tool provided by Ansible to share, discover, and download reusable Ansible roles and collections
- It simplifies the process of reusing and distributing automation code, enabling collaboration and faster development
- **Key Features of Ansible Galaxy**
 - **Role and Collection Repository**
 - Hosts a vast library of roles and collections contributed by the Ansible community and vendors
 - Pre-built automation solutions for tasks like configuring servers, deploying applications, and managing cloud infrastructure
 - **Galaxy Command-Line Tool**
 - The `ansible-galaxy` command is used to install, create, and manage roles and collections
 - **Role/Collection Dependencies**
 - Allows roles and collections to specify dependencies, which are automatically resolved during installation
 - **Collaboration and Sharing**
 - Developers can upload and share their roles/collections with the community or keep them private
 - **Quality Standards**
 - Includes a "quality score" system to help users evaluate the reliability and completeness of shared roles

Ansible Tower



- Ansible Tower is a commercial product developed by Red Hat, built on top of the open-source AWX project
- It is designed for enterprises that require advanced features, support, and integration with other Red Hat products
- **Key Features of Ansible Tower**
 - **Web-based GUI**
 - Manage playbooks, inventories, and workflows via an intuitive interface
 - **Role-based Access Control (RBAC)**
 - Fine-grained permissions for users and teams
 - **Centralized Logging and Auditing**
 - Tracks who ran what tasks and when, ensuring compliance
 - **Workflow Automation**
 - Link multiple playbooks together to create workflows
 - **Scheduling**
 - Automate tasks to run at specific times using a built-in scheduler
 - **Job Templates**
 - Predefine tasks for easy reuse
 - **Notifications**
 - Send notifications to teams via email, Slack, or other systems upon job completion or failure
 - **API Integration**
 - Expose a REST API for integration with CI/CD pipelines and other tools
 - **Scalability**
 - Manage thousands of nodes with clustering and load balancing